# HWK#2 Part A & Part B

Shreesh Karjagi

## Problem 1 (Forward-Backward Algorithm in HMM)

We have an HMM with $T$ time steps, hidden states $z_t$ and observations $x_t$. We derive the forward-backward algorithm to compute $p(z_t = j \mid x_{1,\ldots,T})$ using Variable Elimination.

The HMM joint distribution factors as:

$$p(z_{1:T}, x_{1:T}) = p(z_1)\, p(x_1|z_1) \prod_{t=2}^{T} p(z_t|z_{t-1})\, p(x_t|z_t)$$

Starting from $p(z_t = j \mid x_{1:T}) \propto \alpha_t(j)\, \beta_t(j)$:

### Forward Pass (Eliminate $z_1, \ldots, z_{t-1}$ Left-to-Right)

Define message $m_1(z_2) = \sum_{z_1} p(z_1)\, p(x_1|z_1)\, p(z_2|z_1)$.

For $k = 2, \ldots, t-1$:

$$m_k(z_{k+1}) = \sum_{z_k} m_{k-1}(z_k) \cdot p(x_k|z_k) \cdot p(z_{k+1}|z_k)$$

The forward variable is $\alpha_t(j) = m_{t-1}(z_t{=}j) \cdot p(x_t \mid z_t{=}j)$, giving the recursion:

$$\begin{aligned}
\alpha_1(j) &= p(z_1 = j) \cdot p(x_1 \mid z_1 = j) \\
\alpha_t(j) &= p(x_t \mid z_t{=}j) \sum_i p(z_t{=}j \mid z_{t-1}{=}i) \cdot \alpha_{t-1}(i) \quad \text{for } t = 2, \ldots, T
\end{aligned}$$

### Backward Pass (Eliminate $z_T, \ldots, z_{t+1}$ Right-to-Left)

Define $\tilde{m}_T(z_{T-1}) = \sum_{z_T} p(x_T|z_T)\, p(z_T|z_{T-1})$.

For $k = T{-}1, \ldots, t{+}1$:

$$\tilde{m}_k(z_{k-1}) = \sum_{z_k} \tilde{m}_{k+1}(z_k) \cdot p(x_k|z_k) \cdot p(z_k|z_{k-1})$$

The backward variable is $\beta_t(j) = \tilde{m}_{t+1}(z_t{=}j)$:

$$\begin{aligned}
\beta_T(j) &= 1 \\
\beta_t(j) &= \sum_k p(z_{t+1}{=}k \mid z_t{=}j) \cdot p(x_{t+1} \mid z_{t+1}{=}k) \cdot \beta_{t+1}(k) \quad \text{for } t = T{-}1, \ldots, 1
\end{aligned}$$

### Combining

After both elimination passes, the remaining factor at $z_t$ is $\alpha_t(j) \cdot \beta_t(j) = p(z_t{=}j, x_{1:T})$. Normalizing:

$$\boxed{p(z_t = j \mid x_{1:T}) = \frac{\alpha_t(j) \cdot \beta_t(j)}{\sum_{j'} \alpha_t(j') \cdot \beta_t(j')}}$$

The forward pass computes $\alpha_t$ for all $t$ in $O(TK^2)$ time and the backward pass computes $\beta_t$ for all $t$ in $O(TK^2)$ time (where $K$ is the number of hidden states), giving $O(TK^2)$ total.

# Problem 2 (Inference in a Fully Observed Bayesian Network)

## Graph Structure

From Figure 1 the directed GM has 12 binary variables (0–11). Node 0 (IsSummer) is the root with children $\{1, 2, 3, 4\}$. Diseases (1–4) are parents of symptom nodes (5–11):

| Node | Name | Parents |
|---|---|---|
| 0 | IsSummer | $\emptyset$ |
| 1 | HasFlu | $\{0\}$ |
| 2 | HasFoodPoisoning | $\{0\}$ |
| 3 | HasHayFever | $\{0\}$ |
| 4 | HasPneumonia | $\{0\}$ |
| 5 | HasRespiratoryProblems | $\{1, 3, 4\}$ |
| 6 | HasGastricProblems | $\{1, 2, 4\}$ |
| 7 | HasRash | $\{1, 2, 3\}$ |
| 8 | Coughs | $\{1, 2, 3, 4\}$ |
| 9 | IsFatigued | $\{1, 2, 3, 4\}$ |
| 10 | Vomits | $\{1, 2, 3, 4\}$ |
| 11 | HasFever | $\{1, 2, 3, 4\}$ |

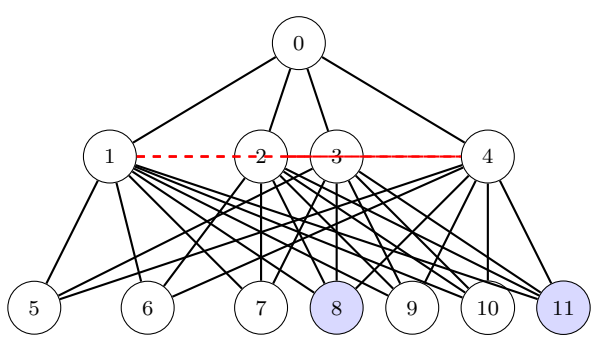
Three queries:

- **Query 1:** $P(\text{HasFlu} \mid \text{HasFever}{=}1, \text{Coughs}{=}1)$
- **Query 2:** $P(\text{HasRash}, \text{Coughs}, \text{IsFatigued}, \text{Vomits}, \text{HasFever} \mid \text{HasPneumonia}{=}1)$
- **Query 3:** $P(\text{Vomits} \mid \text{IsSummer}{=}1)$

## Part (a): Variable Elimination for Query 1

### Moralized Graph

To moralize, we marry all co-parents. Since nodes 8–11 each have parents $\{1, 2, 3, 4\}$, all pairs among $\{1, 2, 3, 4\}$ get moral edges. Similarly node 5 marries $\{1, 3, 4\}$, node 6 marries $\{1, 2, 4\}$, and node 7 marries $\{1, 2, 3\}$. After moralization, $\{1, 2, 3, 4\}$ forms a complete subgraph. Then we drop all edge directions.




Shaded = evidence nodes $(X_8, X_{11})$. Red dashed = moral edges.

**Elimination Ordering and Intermediate Factors**

After applying evidence ($X_8$=1, $X_{11}$=1), the factors that remain non-trivially are:

$$f_0 = P(X_0),\ f_1 = P(X_1|X_0),\ f_2 = P(X_2|X_0),\ f_3 = P(X_3|X_0),\ f_4 = P(X_4|X_0)$$
$$f_8^e = P(X_8{=}1 \mid X_1, X_2, X_3, X_4), \quad f_{11}^e = P(X_{11}{=}1 \mid X_1, X_2, X_3, X_4)$$

Unobserved symptom nodes ($X_5, X_6, X_7, X_9, X_{10}$) each appear in only their own CPD. Since $\sum_{x_i} P(X_i \mid \text{parents}) = 1$, eliminating them produces trivial factors.

**Elimination order:** $X_5, X_6, X_7, X_9, X_{10}, X_0, X_4, X_3, X_2$ (leaving query $X_1$).

1. **Eliminate $X_5$:** $\sum_{X_5} P(X_5|X_1, X_3, X_4) = 1$. Trivial.
2. **Eliminate $X_6, X_7, X_9, X_{10}$:** Same reasoning; each sums to 1.
3. **Eliminate $X_0$:** Multiply $f_0, f_1, f_2, f_3, f_4$, sum out $X_0$:

$$\tau_0(X_1, X_2, X_3, X_4) = \sum_{X_0} P(X_0)\, P(X_1|X_0)\, P(X_2|X_0)\, P(X_3|X_0)\, P(X_4|X_0)$$

   *New factor scope:* $\{X_1, X_2, X_3, X_4\}$, *size* $2^4 = 16$.

4. **Combine with evidence factors:**

$$\phi(X_1, X_2, X_3, X_4) = \tau_0 \cdot f_8^e \cdot f_{11}^e$$

5. **Eliminate $X_4$:** $\tau_4(X_1, X_2, X_3) = \sum_{X_4} \phi(X_1, X_2, X_3, X_4)$. Scope size 8.
6. **Eliminate $X_3$:** $\tau_3(X_1, X_2) = \sum_{X_3} \tau_4(X_1, X_2, X_3)$. Scope size 4.
7. **Eliminate $X_2$:** $\tau_2(X_1) = \sum_{X_2} \tau_3(X_1, X_2)$. Scope size 2.

Normalize: $P(X_1 \mid \text{evidence}) = \tau_2(X_1) / \sum_{x_1} \tau_2(x_1)$.

**Complexity:** Largest intermediate factor has scope $\{X_1, X_2, X_3, X_4\}$, so size $2^4 = 16$. Time: $O(2^5)$ per elimination step (multiply + sum). Space: $O(2^4)$.

## Part (b): Message Passing

### (i) Clique Tree View (Query 1)

Using the same elimination ordering, the non-trivial cliques formed are:

- $C_1 = \{X_0, X_1, X_2, X_3, X_4\}$ — from eliminating $X_0$
- $C_2 = \{X_1, X_2, X_3, X_4\}$ — hosts the evidence factors $f_8^e, f_{11}^e$

Since the symptom eliminations all produce trivial factors, the effective clique tree is:

$$C_1\{X_0, X_1, X_2, X_3, X_4\} \ \text{--}\ C_2\{X_1, X_2, X_3, X_4\}$$

with separator $S_{12} = \{X_1, X_2, X_3, X_4\}$.

**Messages:**

$$\delta_{1\to 2}(X_1, X_2, X_3, X_4) = \sum_{X_0} P(X_0)\, P(X_1|X_0)\, P(X_2|X_0)\, P(X_3|X_0)\, P(X_4|X_0)$$

The belief at $C_2$:

$$\beta_2(X_1, X_2, X_3, X_4) = \delta_{1\to 2} \cdot P(X_8{=}1|X_1, X_2, X_3, X_4) \cdot P(X_{11}{=}1|X_1, X_2, X_3, X_4)$$

Marginalize to get $P(X_1|\text{evidence}) \propto \sum_{X_2,X_3,X_4} \beta_2$.

Complexity same as VE: largest clique $|C_1| = 5$ variables, so time $O(2^5)$, space $O(2^5)$.

### (ii) Factor Graph View (Query 1)

On the factor graph, factors are CPDs and evidence indicators. Three example variable-to-factor messages:

$$\begin{aligned}
\mu_{X_0\to f_1}(X_0) &= f_0(X_0) \cdot \mu_{f_2\to X_0}(X_0) \cdot \mu_{f_3\to X_0}(X_0) \cdot \mu_{f_4\to X_0}(X_0) \\
\mu_{X_3\to f_8^e}(X_3) &= \mu_{f_3\to X_3}(X_3) \cdot \mu_{f_7\to X_3}(X_3) \\
\mu_{X_4\to f_{11}^e}(X_4) &= \mu_{f_4\to X_4}(X_4) \cdot \mu_{f_5\to X_4}(X_4) \cdot \mu_{f_6\to X_4}(X_4)
\end{aligned}$$

Three example factor-to-variable messages:

$$\begin{aligned}
\mu_{f_1\to X_1}(X_1) &= \sum_{X_0} P(X_1|X_0) \cdot \mu_{X_0\to f_1}(X_0) \\
\mu_{f_8^e\to X_1}(X_1) &= \sum_{X_2,X_3,X_4} P(X_8{=}1|X_1, X_2, X_3, X_4) \cdot \mu_{X_2\to f_8^e} \cdot \mu_{X_3\to f_8^e} \cdot \mu_{X_4\to f_8^e} \\
\mu_{f_{11}^e\to X_1}(X_1) &= \sum_{X_2,X_3,X_4} P(X_{11}{=}1|X_1, X_2, X_3, X_4) \cdot \mu_{X_2\to f_{11}^e} \cdot \mu_{X_3\to f_{11}^e} \cdot \mu_{X_4\to f_{11}^e}
\end{aligned}$$

Complexity: factor $f_8$ has scope $\{X_1, X_2, X_3, X_4\}$ so messages cost $O(2^4)$. Overall: $O(2^4 \cdot K)$ time, $O(2^4)$ space.

## Part (c): Parameter Estimation

Parameters estimated from `dataset.dat` (4,000,000 samples) via maximum likelihood:

- Root node 0: $\hat{P}(X_0 = v) = \text{Count}(X_0{=}v)/N$
- Other nodes: $\hat{P}(X_i{=}v \mid \pi(X_i){=}\mathbf{u}) = \text{Count}(X_i{=}v,\ \pi(X_i){=}\mathbf{u})\ /\ \text{Count}(\pi(X_i){=}\mathbf{u})$

## Part (d): Model Accuracy

$L_1$ distance from true joint (`joint.dat`): $\sum_{a=0}^{4095} |p(a) - \hat{p}(a)| = \mathbf{0.3308}$
Log-likelihood of data (true dist): $-14{,}392{,}338$ (avg per sample: $-3.598$)
Log-likelihood of data (model): $-15{,}213{,}183$ (avg per sample: $-3.803$)

The model captures the main structure well. The $L_1$ gap of 0.33 arises because the problem states that noise has been added to the true distribution relative to the BN structure. The model's log-likelihood is lower than the true distribution's by about 0.2 nats per sample, reflecting the inability of the factored BN model to capture the added noise correlations. Despite this, the conditional query results (parts a-c) show VE model estimates within 0.01–0.02 of the true joint values, confirming the model is adequate for inference.

## Part (e): Query Results (VE Implementation)

**Query 1:** $P(\text{HasFlu} \mid \text{HasFever=1}, \text{Coughs=1})$

| | True Joint | VE Model | Sample Counting |
|---|---|---|---|
| $P$(HasFlu=0) | 0.4838 | 0.4838 | 0.4828 |
| $P$(HasFlu=1) | 0.5162 | 0.5162 | 0.5172 |

VE time: 0.003s  Sample counting: 0.03s

**Query 2:** $P(\text{Symptoms} \mid \text{HasPneumonia=1})$
Top 3 most likely (HasRash, Coughs, IsFatigued, Vomits, HasFever):

| **Combination** | **VE Model** | **True Joint** |
|---|---|---|
| (F, T, T, T, T) | 0.1303 | 0.1455 |
| (F, T, T, T, F) | 0.1205 | 0.1347 |
| (F, F, T, T, T) | 0.0841 | 0.0876 |

VE time: 0.008s

**Query 3:** $P(\text{Vomits} \mid \text{IsSummer=1})$

| | True Joint | VE Model | Sample Counting |
|---|---|---|---|
| $P$(Vomits=0) | 0.9026 | 0.9029 | 0.9027 |
| $P$(Vomits=1) | 0.0974 | 0.0971 | 0.0973 |

VE time: 0.028s  Sample counting: 0.04s

Variable elimination produces results very close to both the true joint and sample counting estimates.

## Discussion

The BN model was fit to 4,000,000 samples using maximum likelihood estimation of all CPDs. The resulting model's joint distribution has an $L_1$ distance of 0.3308 from the true joint (`joint.dat`), meaning on average the model probability for each of the 4096 configurations differs from the truth by about $8 \times 10^{-5}$. This gap is expected since the problem notes that noise was added to the true distribution beyond what the BN structure can capture.

The log-likelihood of the data under the true distribution is −14,392,338 (avg −3.598 per sample), while under the learned model it is −15,213,183 (avg −3.803 per sample). The difference of about 0.2 nats per sample reflects the model's structural limitation in representing the noisy components of the true distribution.

Despite this gap at the joint level, the conditional query results are encouraging. For Query 1, the VE model estimate of $P(\text{HasFlu=1} \mid \text{evidence})$ matches the true joint to four decimal places (0.5162). Query 3 similarly shows agreement within 0.0003. Even Query 2, which involves a 5-variable conditional distribution, shows the same top-3 most likely symptom combinations with probabilities within 0.01–0.02 of the true values. This indicates that while the model's joint is imperfect, its conditional inference is quite reliable.

Sample counting from the dataset produces results consistent with both VE and the true joint, validating the implementation. VE runs in milliseconds compared to sample counting, confirming the efficiency advantage of exact inference over naïve enumeration.

**Code:**

```
import numpy as np
from collections import defaultdict
import time

def load_joint(filepath):
    joint = {}
    with open(filepath, 'r') as f:
        for line in f:
            parts = line.strip().split('\t')
            joint[int(parts[0])] = float(parts[1])
    return joint

def load_dataset(filepath):
    samples = []
    with open(filepath, 'r') as f:
        for line in f:
            samples.append(int(line.strip()))
    return samples

def int_to_bits(n, num_bits=12):
    return [(n >> i) & 1 for i in range(num_bits)]
parents = {
    0: [], # IsSummer
    1: [0], # HasFlu
    2: [0], # HasFoodPoisoning
    3: [0], # HasHayFever
    4: [0], # HasPneumonia
    5: [1, 3, 4], # HasRespiratoryProblems
    6: [1, 2, 4], # HasGastricProblems
    7: [1, 2, 3], # HasRash
    8: [1, 2, 3, 4], # Coughs
    9: [1, 2, 3, 4], # IsFatigued
    10: [1, 2, 3, 4],# Vomits
    11: [1, 2, 3, 4] # HasFever
}

def estimate_parameters(samples):
    num_vars = 12
    assignment_counts = defaultdict(int)
    for s in samples:
        assignment_counts[s] += 1

    cpds = {}
    for var in range(num_vars):
        par = parents[var]
        counts = defaultdict(lambda: [0, 0])
```

```
        for s, cnt in assignment_counts.items():
            bits = int_to_bits(s, num_vars)
            parent_vals = tuple(bits[p] for p in par)
            counts[parent_vals][bits[var]] += cnt

        cpd = {}
        for parent_vals, c in counts.items():
            total = c[0] + c[1]
            cpd[parent_vals] = [c[0]/total, c[1]/total] if total > 0
                else [0.5, 0.5]
        cpds[var] = cpd
    return cpds

def compute_model_joint(cpds):
    model_joint = {}
    for a in range(4096):
        bits = int_to_bits(a, 12)
        prob = 1.0
        for var in range(12):
            pv = tuple(bits[p] for p in parents[var])
            prob *= cpds[var].get(pv, [0.5, 0.5])[bits[var]]
        model_joint[a] = prob
    return model_joint

class Factor:
    def __init__(self, variables, values):
        self.variables = sorted(variables)
        self.values = values

def multiply_factors(f1, f2):
    all_vars = sorted(set(f1.variables + f2.variables))
    new_values = {}
    for ai in range(2**len(all_vars)):
        asgn = {v: (ai >> i) & 1 for i, v in enumerate(all_vars)}
        k1 = tuple(asgn[v] for v in f1.variables)
        k2 = tuple(asgn[v] for v in f2.variables)
        kn = tuple(asgn[v] for v in all_vars)
        new_values[kn] = f1.values.get(k1, 0) * f2.values.get(k2, 0)
    return Factor(all_vars, new_values)

def sum_out(factor, var):
    if var not in factor.variables:
        return factor
    new_vars = [v for v in factor.variables if v != var]
    new_values = defaultdict(float)
    for key, val in factor.values.items():
        new_key = tuple(k for i, k in enumerate(key) if factor.
           variables[i] != var)
```

```
        new_values[new_key] += val
    return Factor(new_vars, dict(new_values))

def create_cpd_factor(var, cpds):
    par = parents[var]
    all_vars = sorted(par + [var])
    values = {}
    for ai in range(2**len(all_vars)):
        asgn = {v: (ai >> i) & 1 for i, v in enumerate(all_vars)}
        pv = tuple(asgn[p] for p in par)
        values[tuple(asgn[v] for v in all_vars)] = cpds[var].get(pv,
            [0.5,0.5])[asgn[var]]
    return Factor(all_vars, values)

def variable_elimination(cpds, query_vars, evidence):
    factors = [create_cpd_factor(var, cpds) for var in range(12)]

    for evar, eval_ in evidence.items():
        new_factors = []
        for f in factors:
            if evar in f.variables:
                nv = {}
                new_vars = [v for v in f.variables if v != evar]
                for key, val in f.values.items():
                    if key[f.variables.index(evar)] == eval_:
                        nk = tuple(k for i, k in enumerate(key) if f
                            .variables[i] != evar)
                        nv[nk] = val
                new_factors.append(Factor(new_vars, nv))
            else:
                new_factors.append(f)
        factors = new_factors
    elim_order = [v for v in range(12) if v not in query_vars and v
       not in evidence]
    for var in elim_order:
        involved = [f for f in factors if var in f.variables]
        remaining = [f for f in factors if var not in f.variables]
        if not involved:
            continue
        product = involved[0]
        for f in involved[1:]:
            product = multiply_factors(product, f)
        factors = remaining + [sum_out(product, var)]

    result = factors[0]
    for f in factors[1:]:
        result = multiply_factors(result, f)
```

```
    total = sum(result.values.values())
    if total > 0:
        result.values = {k: v/total for k, v in result.values.items
           ()}
    return result

def query_true_joint(joint, query_vars, evidence):
    result = defaultdict(float)
    for a, prob in joint.items():
        bits = int_to_bits(a, 12)
        if all(bits[ev] == val for ev, val in evidence.items()):
            result[tuple(bits[qv] for qv in query_vars)] += prob
    total = sum(result.values())
    return {k: v/total for k, v in result.items()} if total > 0 else
        result

def get_bit(sample, var):
    return (sample >> var) & 1

if __name__ == "__main__":
    data_dir = '/Users/skarjagi6/Library/CloudStorage/Dropbox-GaTech
       /Shreesh␣Karjagi/coursework/GRAPH_ML/HWK2/Data-HWK2/Data-
       Problem-2'
    joint = load_joint(f'{data_dir}/joint.dat')
    samples = load_dataset(f'{data_dir}/dataset.dat')
    print(f"Loaded␣joint.dat:␣{len(joint)}␣entries,␣sum={sum(joint.
       values()):.6f}")
    print(f"Loaded␣dataset.dat:␣{len(samples)}␣samples")

   #part c
    cpds = estimate_parameters(samples)
    model_joint = compute_model_joint(cpds)

   #part d
    l1 = sum(abs(joint.get(i,0) - model_joint.get(i,0)) for i in
       range(4096))
    ll_true = sum(np.log(max(joint.get(s,1e-30),1e-30)) for s in
       samples)
    ll_model = sum(np.log(max(model_joint.get(s,1e-30),1e-30)) for s
        in samples)
    print(f"\nL1␣distance:␣{l1:.6f}")
    print(f"Log-lik␣(true):␣{ll_true:.2f}␣(avg␣{ll_true/len(samples)
       :.6f})")
    print(f"Log-lik␣(model):␣{ll_model:.2f}␣(avg␣{ll_model/len(
       samples):.6f})")

   #part e
    queries = [
```

```
        ("Q1: P(HasFlu | HasFever=T, Coughs=T)", [1], {11:1, 8:1}),
        ("Q2: P(HasRash,Coughs,Fatigued,Vomits,Fever | Pneumonia=T)"
           ,
         [7,8,9,10,11], {4:1}),
        ("Q3: P(Vomits | IsSummer=T)", [10], {0:1}),
    ]

    for name, qvars, ev in queries:
        print(f"\n{name}")
        true_r = query_true_joint(joint, qvars, ev)
        t0 = time.time()
        ve_r = variable_elimination(cpds, qvars, ev)
        t_ve = time.time() - t0
        print(f"True joint: {dict(sorted(true_r.items()))}")
        print(f"VE model:   {dict(sorted(ve_r.values.items()))}")
        print(f"VE time: {t_ve:.4f}s")

    samples_arr = np.array(samples)

    t0 = time.time()
    mask_q1 = (samples_arr >> 11 & 1 == 1) & (samples_arr >> 8 & 1
       == 1)
    matching = samples_arr[mask_q1]
    flu1 = np.mean((matching >> 1) & 1)
    t_sc1 = time.time() - t0
    print(f"\nQ1 sample counting: P(HasFlu=0)={1-flu1:.4f}, P(HasFlu
       =1)={flu1:.4f}, time: {t_sc1:.2f}s")

    t0 = time.time()
    mask_q3 = (samples_arr >> 0 & 1 == 1)
    matching3 = samples_arr[mask_q3]
    vom1 = np.mean((matching3 >> 10) & 1)
    t_sc3 = time.time() - t0
    print(f"Q3 sample counting: P(Vomits=0)={1-vom1:.4f}, P(Vomits
       =1)={vom1:.4f}, time: {t_sc3:.2f}s")
```

## Problem 3 (Maximum Probable Explanation)

Pairwise Markov network on 100 binary variables forming a cycle:

$$P(\mathbf{x}) = \varphi(x_1, x_{100}) \prod_{i=1}^{99} \varphi(x_i, x_{i+1})$$

### Part A: Computing $\arg\max_{x_1,\ldots,x_{100}} P(\mathbf{x})$

The graph is a cycle. To compute MAP efficiently, we break the cycle by conditioning on $x_1$.

**Algorithm:**

1. For each value of $x_1 \in \{0, 1\}$, the remaining structure is a chain.
2. Run **max-product** (Viterbi) along $x_2, \ldots, x_{100}$:

$$\begin{aligned} \delta_2(x_2) &= \varphi(x_1, x_2) \\ \delta_i(x_i) &= \max_{x_{i-1}} \left[\delta_{i-1}(x_{i-1}) \cdot \varphi(x_{i-1}, x_i)\right], \quad i = 3, \ldots, 100 \end{aligned}$$

3. At the end close the cycle: $\delta^*_{100}(x_{100}) = \delta_{100}(x_{100}) \cdot \varphi(x_{100}, x_1)$.
4. Backtrack to recover optimal $x^*_{100}, \ldots, x^*_2$.
5. Pick the $x_1$ giving larger $\max_{x_{100}} \delta^*_{100}$.

**Complexity:** $O(2 \times 100 \times 4) = O(800)$, linear in the number of variables.

### Part B: Junction Tree

The cycle graph on 100 variables is not a tree. To form a valid junction tree, we triangulate by connecting $x_1$ to all other nodes, creating size-3 cliques:

**Junction tree** (chain of 98 cliques):

$$\underbrace{\{x_1, x_2, x_3\}}_{C_1} -- \underbrace{\{x_1, x_3, x_4\}}_{C_2} -- \cdots -- \underbrace{\{x_1, x_{99}, x_{100}\}}_{C_{98}}$$

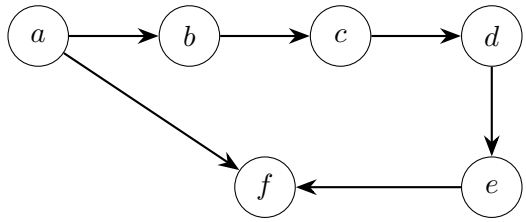
Separators: $S_k = \{x_1, x_{k+2}\}$ between $C_k$ and $C_{k+1}$.
**Factor assignment:**

- $C_1 = \{x_1, x_2, x_3\}$: assign $\varphi(x_1, x_2) \cdot \varphi(x_2, x_3)$
- $C_k = \{x_1, x_{k+1}, x_{k+2}\}$ for $k = 2, \ldots, 97$: assign $\varphi(x_{k+1}, x_{k+2})$
- $C_{98} = \{x_1, x_{99}, x_{100}\}$: assign $\varphi(x_{99}, x_{100}) \cdot \varphi(x_{100}, x_1)$

## Problem 4 (Clique Trees)

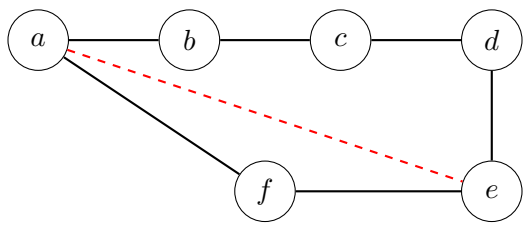
Distribution: $p(a, b, c, d, e, f) = p(a)\, p(b|a)\, p(c|b)\, p(d|c)\, p(e|d)\, p(f|a, e)$

### (a) Belief Network




### (b) Moralized Graph

Node $f$ has parents $\{a, e\}$, so we add the moral edge $a - e$. No other node has multiple parents. Drop all directions.



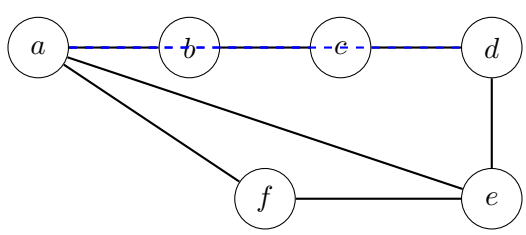

Red dashed = moral edge $a - e$.

### (c) Triangulated Graph

The moralized graph has a 5-cycle $a{-}b{-}c{-}d{-}e{-}a$ (edges: $a{-}b, b{-}c, c{-}d, d{-}e, e{-}a$). This is chordless, so we need fill edges.

**Elimination ordering:** $f, b, c, d, e, a$

1. Eliminate $f$: neighbors $\{a, e\}$ already connected. No fill.
2. Eliminate $b$: neighbors $\{a, c\}$. **Add fill** $a{-}c$.
3. Eliminate $c$: neighbors $\{a, d\}$ (using new $a{-}c$ and existing $c{-}d$). **Add fill** $a{-}d$.
4. Eliminate $d$: neighbors $\{a, e\}$ already connected. No fill.
5. Eliminate $e, a$: no fills needed.




Blue dashed = fill edges. All maximal cliques have size 3.

### (d) Junction (Clique) Tree

Maximal cliques: $C_1 = \{a, b, c\}$, $C_2 = \{a, c, d\}$, $C_3 = \{a, d, e\}$, $C_4 = \{a, e, f\}$.

$$\boxed{\{a,b,c\}} \overset{\{a,c\}}{\text{———}} \boxed{\{a,c,d\}} \overset{\{a,d\}}{\text{———}} \boxed{\{a,d,e\}} \overset{\{a,e\}}{\text{———}} \boxed{\{a,e,f\}}$$

**Running Intersection Property:** $a$: in all 4 cliques (contiguous). $b$: only in $C_1$. $c$: in $C_1, C_2$ (contiguous). $d$: in $C_2, C_3$ (contiguous). $e$: in $C_3, C_4$ (contiguous). $f$: only in $C_4$. ✓

### (e) Clique Potential Initialization

Assign each factor to a clique containing its scope:

$$\begin{aligned}
\psi_1(a, b, c) &= p(a) \cdot p(b|a) \cdot p(c|b) \\
\psi_2(a, c, d) &= p(d|c) \\
\psi_3(a, d, e) &= p(e|d) \\
\psi_4(a, e, f) &= p(f|a, e)
\end{aligned}$$

### (f) Message Algorithm for $p(a \mid e = e_0)$

Apply evidence $e = e_0$: reduce $\psi_3(a, d) \leftarrow p(e_0|d)$ and $\psi_4(a, f) \leftarrow p(f|a, e_0)$.

Root the tree at $C_1$ (contains query variable $a$). Collect messages toward $C_1$:

**Message** $C_4 \to C_3$ over separator $\{a\}$ (since $e$ is fixed):

$$m_{4\to3}(a) = \sum_f p(f \mid a, e_0) = 1$$

**Message** $C_3 \to C_2$ over separator $\{a, d\}$:
$C_3$ belief: $\psi_3(a, d) \cdot m_{4\to3}(a) = p(e_0|d) \cdot 1$. Sum out variables in $C_3 \setminus S_{23}$. Since $e$ is already fixed and $C_3 = \{a, d, e\}$, the remaining variables in $C_3$ beyond separator $\{a, d\}$ is just $e$ which is observed. So:

$$m_{3\to2}(a, d) = p(e_0 \mid d)$$

**Message** $C_2 \to C_1$ over separator $\{a, c\}$:
$C_2$ belief: $\psi_2(a, c, d) \cdot m_{3\to2}(a, d) = p(d|c) \cdot p(e_0|d)$. Sum out $d$:

$$m_{2\to1}(a, c) = \sum_d p(d|c) \cdot p(e_0|d)$$

**Final belief at $C_1$:**

$$\beta_1(a, b, c) = \psi_1(a, b, c) \cdot m_{2\to1}(a, c) = p(a)\, p(b|a)\, p(c|b) \cdot \sum_d p(d|c)\, p(e_0|d)$$

$$p(a \mid e{=}e_0) \propto \sum_{b,c} p(a)\, p(b|a)\, p(c|b) \cdot \left[\sum_d p(d|c)\, p(e_0|d)\right]$$

## Problem 5 (Partition Function in MRF)

Binary MRF on $n \times n$ lattice: $p(\mathbf{x}) = Z^{-1} \prod_{i>j,\,\text{neighbors}} \phi(x_i, x_j)$ where $\phi(x_i, x_j) = e^{\mathbb{I}[x_i = x_j]}$.

### Cluster Representation

Stack the $t$-th column's variables into cluster $X_t = (x_{1,t}, \ldots, x_{n,t})$. The graph becomes a chain $X_1 - X_2 - \cdots - X_n$. Each cluster has $2^n$ states. The potential between adjacent clusters separates into horizontal edges (between columns) and vertical edges (within a column):

$$\Psi_t(X_t) = \prod_{i=1}^{n-1} e^{\mathbb{I}[x_{i,t} = x_{i+1,t}]}, \qquad T(X_t, X_{t+1}) = \prod_{i=1}^{n} e^{\mathbb{I}[x_{i,t} = x_{i,t+1}]}$$

### Complexity

**Naive:** Message passing on the chain requires matrix-vector products of size $2^n \times 2^n$, costing $O(n \cdot 2^{2n})$.

**Butterfly optimization:** The transfer matrix $T(X_t, X_{t+1})$ *factors* as a product of $n$ independent $2 \times 2$ transfers (one per row):

$$T(X_t, X_{t+1}) = \prod_{i=1}^{n} e^{\mathbb{I}[x_{i,t} = x_{i,t+1}]}$$

Each $2 \times 2$ transfer $\begin{pmatrix} e & 1 \\ 1 & e \end{pmatrix}$ can be applied to pairs of states differing in bit $i$, sweeping through all $n$ bits. This gives an $O(n \cdot 2^n)$ matrix-vector product instead of $O(2^{2n})$.

**Total complexity:** $O(n^2 \cdot 2^n)$. For $n = 20$: $O(400 \cdot 2^{20}) \approx 4 \times 10^8$ — tractable.

### Validation

| $n$ | $\log Z$ (brute-force) | $\log Z$ (butterfly) |
|---|---|---|
| 2 | 5.2976 | 5.2976 |
| 3 | 13.8915 | 13.8915 |
| 4 | 26.4977 | 26.4977 |
| 5 | 43.1269 | 43.1269 |

$\log Z$ for $n = 20$: **777.5118**
Computation time: ~113 seconds.

**Code:**

```
import numpy as np
import time

def compute_log_partition_naive(n):
    num_states = 2**n
    def intra(s):
        e = 0
        for i in range(n-1):
            if ((s>>i)&1) == ((s>>(i+1))&1): e += 1
```

```
        return e
    def inter(s1, s2):
        e = 0
        for i in range(n):
            if ((s1>>i)&1) == ((s2>>i)&1): e += 1
        return e

    log_msg = np.array([float(intra(s)) for s in range(num_states)])
    for col in range(1, n):
        new_log_msg = np.full(num_states, -np.inf)
        for s2 in range(num_states):
            terms = np.array([log_msg[s1] + inter(s1, s2) for s1 in
               range(num_states)])
            mx = np.max(terms)
            new_log_msg[s2] = mx + np.log(np.sum(np.exp(terms - mx))
               ) + intra(s2)
        log_msg = new_log_msg
    mx = np.max(log_msg)
    return mx + np.log(np.sum(np.exp(log_msg - mx)))

def compute_log_partition_butterfly(n, verbose=False):
    """
␣␣␣␣butterfly␣transfer␣method,␣horizontal␣transfer␣factors␣as␣n␣
   independent␣2x2␣transfers
␣␣␣␣so␣matrix-vector␣product␣is␣O(n␣*␣2^n)␣instead␣of␣O(2^{2n})
␣␣␣␣"""
    num_states = 2**n
    e_same = np.exp(1.0) # phi(a,b)=e when a==b
    e_diff = 1.0 # phi(a,b)=1 when a!=b

    intra_log = np.zeros(num_states)
    for s in range(num_states):
        for i in range(n - 1):
            if ((s >> i) & 1) == ((s >> (i + 1)) & 1):
                intra_log[s] += 1.0

    log_msg = intra_log.copy()

    for col in range(1, n):
        if verbose and col % 5 == 0:
            print(f"␣␣Column␣{col+1}/{n}")
        max_log = np.max(log_msg)
        msg = np.exp(log_msg - max_log)

        #apply n independent 2x2 transfers
        for i in range(n):
            temp = msg.copy()
            mask = 1 << i
```

```
            for s in range(num_states):
                if s & mask == 0:
                    s1 = s | mask
                    msg[s]  = temp[s] * e_same + temp[s1] * e_diff
                    msg[s1] = temp[s] * e_diff + temp[s1] * e_same

        log_msg = np.log(msg + 1e-300) + max_log + intra_log

    mx = np.max(log_msg)
    return mx + np.log(np.sum(np.exp(log_msg - mx)))

if __name__ == "__main__":
    print("Validation␣(naive␣vs␣butterfly):")
    for test_n in [2, 3, 4, 5]:
        ln = compute_log_partition_naive(test_n)
        lb = compute_log_partition_butterfly(test_n)
        print(f"␣␣n={test_n}:␣naive={ln:.6f},␣butterfly={lb:.6f},␣
           diff={abs(ln-lb):.2e}")

    #n=20
    print(f"\nComputing␣log(Z)␣for␣n=20")
    t0 = time.time()
    logZ = compute_log_partition_butterfly(20, verbose=True)
    print(f"\nlog(Z)␣for␣n=20␣=␣{logZ:.6f}")
    print(f"Time:␣{time.time()-t0:.1f}s")
```

## Problem 6 (Bear/Bull Market)

### Formulation

This is a Hidden Markov Model:

- **Hidden state** $S_t \in \{\text{bear}(0), \text{bull}(1)\}$: market regime
- **Observed** $P_t \in \{1, \ldots, 100\}$: asset price
- $T = 200$ time steps

**Market state transitions:**

$$A_{ij} = P(S_t{=}j \mid S_{t-1}{=}i) : \quad \begin{pmatrix} P(\text{bear} \to \text{bear}) & P(\text{bear} \to \text{bull}) \\ P(\text{bull} \to \text{bear}) & P(\text{bull} \to \text{bull}) \end{pmatrix} = \begin{pmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{pmatrix}$$

**Price transitions:** Given market state $S_{t-1} = s$, the price changes from $P_{t-1}$ to $P_t$ according to the $100 \times 100$ matrix `pbear` (if $s = \text{bear}$) or `pbull` (if $s = \text{bull}$).

**Priors:** $P(S_1)$ uniform, $P(P_1)$ uniform over $\{1, \ldots, 100\}$.

### Forward Algorithm

We run the forward algorithm with state $\alpha_t(s) = P(\text{prices}(1{:}t),\ S_t{=}s)$, normalizing at each step:

$$\alpha_1(s) = P(S_1{=}s) \cdot P(P_1)$$
$$\alpha_t(s') = \sum_s \alpha_{t-1}(s) \cdot P(S_t{=}s' | S_{t-1}{=}s) \cdot P(P_t | P_{t-1}, S_{t-1}{=}s)$$

After processing all $T = 200$ observations:

$$P(S_T{=}\text{bear} \mid \text{data}) = 0.3568, \quad P(S_T{=}\text{bull} \mid \text{data}) = 0.6432$$

The predictive distribution for $P_{T+1}$:

$$P(P_{T+1}{=}p \mid \text{data}) = \sum_s P(S_T{=}s \mid \text{data}) \cdot P(P_{T+1}{=}p \mid P_T, S_T{=}s)$$

### Results

**1. Expected gain:**

$$E[\text{price}(T{+}1) - \text{price}(T) \mid \text{data}] = E[\text{price}(T{+}1)] - 58 = 59.4323 - 58 = \mathbf{1.4323}$$

**2. Standard deviation of gain:**

$$\text{Var}(\text{gain}) = E[P_{T+1}^2] - (E[P_{T+1}])^2 = 3605.15 - 3532.20 = 72.95$$

$$\text{Std}(\text{gain}) = \sqrt{72.95} = \mathbf{8.5410}$$

The positive expected gain is consistent with the higher posterior probability of a bull market at $T = 200$.

**Code:**

```
import numpy as np
import scipy.io

data_dir = '/Users/skarjagi6/Library/CloudStorage/Dropbox-GaTech/
   Shreesh␣Karjagi/coursework/GRAPH_ML/HWK2/Data-Problem-6/
   BearBullproblem.mat'
data = scipy.io.loadmat(data_dir)
pbull = data['pbull'] #100x100: pbull[new, old]
pbear = data['pbear'] #100x100: pbear[new, old]
prices = data['p'].flatten()  # 200 observed prices

T = len(prices)
print(f"T␣=␣{T},␣price␣range:␣[{prices.min()},␣{prices.max()}]")
print(f"Last␣price:␣price(T={T})␣=␣{prices[-1]}")

#market state 0=bear, 1=bull
#market_trans[new_state, old_state]
market_trans = np.array([[0.8, 0.3],   #P(bear|bear), P(bear|bull)
                         [0.2, 0.7]])  #P(bull|bear), P(bull|bull)

price_trans = {0: pbear, 1: pbull}

#Forward algorithm
#alpha[s] = P(prices(1:t), market(t)=s), normalized at each step
alpha = np.array([0.5 * (1.0/100), 0.5 * (1.0/100)]) #prior * P(
   price_1)

for t in range(1, T):
    new_alpha = np.zeros(2)
    cur_idx = prices[t] - 1
    prev_idx = prices[t-1] - 1
    for s_new in range(2):
        for s_old in range(2):
            new_alpha[s_new] += (alpha[s_old]
                * market_trans[s_new, s_old]
                * price_trans[s_old][cur_idx, prev_idx])
    alpha = new_alpha / new_alpha.sum()  # normalize

print(f"\nPosterior␣at␣T:␣P(bear)={alpha[0]:.6f},␣P(bull)={alpha
   [1]:.6f}")

#predict price(T+1)
last_idx = prices[-1] - 1
price_dist = np.zeros(100)
for s in range(2):
    price_dist += alpha[s] * price_trans[s][:, last_idx]
price_dist /= price_dist.sum()
```

```
E_price = sum((p+1) * price_dist[p] for p in range(100))
E_price_sq = sum(((p+1)**2) * price_dist[p] for p in range(100))
var_price = E_price_sq - E_price**2

expected_gain = E_price - prices[-1]
std_gain = np.sqrt(var_price)

print(f"\nE[price(T+1)] = {E_price:.4f}")
print(f"Expected gain = E[price(T+1)] - price(T) = {expected_gain:.4
   f}")
print(f"Std(gain) = {std_gain:.4f}")
```